

DropDash (Temporary File Storage)

A project report

Submitted in partial fulfilment of the requirements for the award of degree of

Bachelor of Technology (Computer Science and Engineering)

Submitted to

LOVELY PROFESSIONAL UNIVERSITY PHAGWARA, PUNJAB

Under the Guidance of

Mr. Rahul Punj: 30317



From January 2024 to May 2024

SUBMITTED BY

Shaik Mahammed Irshad: 12020419

Declaration by student

To whom so ever it may concern

I, **Shaik Mahammed Irshad: 12020419** hereby declare that the work done by us on **DropDash (Temporary File Storage)** under the supervision of **Rahul Punj**, Assistant Professor, Lovely professional University, Phagwara, Punjab, is a record of original work for the partial fulfilment of the requirements for the award of the degree, **B.Tech CSE.**

Name of the Student (Registration Number)

Signature of the student

Shaik Mahammed Irshad: 12020419



Dated: 03 May 2024

Declaration by the supervisor

To whom so ever it may concern

This is to certify that **Shaik Mahammed Irshad: 12020419** from Lovely Professional University, Phagwara, Punjab, has worked on **DropDash (Temporary File Storage)** under my supervision from. It is further stated that the work carried out by the student is a record of original work to the best of my knowledge for the partial fulfilment of the requirements for the award of the degree, degree name.

Name of Supervisor: UID of Supervisor

Rahul Punj: 30317

A handwritten signature in blue ink, appearing to read 'Rahul Punj', is positioned above the printed name of the supervisor.

Signature of Supervisor

Acknowledgement

We would especially like to thank **Mr. Rahul Punj**, our professor, for all his valuable guidance and insightful feedback, which improved our work. His instructions have served as the major contributor towards the completion of the project.

We would want to sincerely thank Lovely Professional University for giving us the chance to work together on the DropDash.

Finally, I would want to express our gratitude to our friends and family for their support and tolerance throughout this effort. We appreciate everyone's help, both direct and indirect, in getting this project finished.

TABLE OF CONTENTS

Cover Page	(i)
PAC form.....	(ii)
Declaration by Students.	(iii)
Declaration by the supervisor.....	(iv)
Acknowledgement.....	(v)
Table of Contents.	(vi)
1. Chapter – 1 Introduction	7
1.1. Purpose	7
1.2. Project Background	7
1.3. Project Overview	9
1.3.1. Problem Statement	9
1.3.2. Requirements	9
1.3.3. Objectives	10
2. Chapter - 2 Review of Literature	11
3. Chapter – 3 Implementation	14
3.1. Tools and Techniques	14
3.2. Technology Stack	15
3.3. Development Process	17
3.3.1. System Design	17
3.3.2. Components and Their Interactions	18
3.3.3. Diagrams Illustrating the Architecture	19
3.4. Frontend Development	20
3.5. Backend Development	22
3.5.1. File Upload	22
3.5.2. Link Generation	24
3.5.3. Expiration Logic	24
3.5.4. Implantation with Azure Services	25
3.6. Authentication and Authorization	37
3.7. Security Measures	38
3.7.1. Data Encryption	38
3.7.2. Access Control	39
4. Chapter – 4 Results and Discussions	41
4.1. Results Overview	41
4.2. Frontend Development Results	42
4.3. Backend Integration Results	44
4.4. Performance Analysis	45
5. Final Chapter - Conclusion and Future Scope	47
5.1. Conclusion	47
5.2. Future Scope	47
6. References	49
7. Plagiarism Report	52

Chapter – 1 Introduction

Have you ever faced the frustration of trying to share large files with colleagues, friends, or family? Traditional methods like email attachments often have strict file size limits, making it impossible to send sizable documents, videos, or multimedia files. And while third-party file-sharing services offer a solution, they come with their own set of concerns – limited storage space, potential security risks, and lack of control over who can access your shared files and for how long.

It was this pain point that sparked the idea for DropDash – a secure, user-friendly, and scalable file-sharing web application designed to make sharing files of any size a breeze, without compromising on privacy or control.

1.1. Purpose

This report serves as documentation for DropDash, a secure file-sharing web application developed as a capstone project for the Lovely Professional University B.Tech CSE. It provides an overview of the project's background, objectives, technical implementation, and future enhancements.

1.2. Project Background

In today's digital age, file sharing has become an integral part of our personal and professional lives. We constantly need to exchange documents, media files, and other data with colleagues, clients, friends, and family. However, the traditional methods of file sharing often fall short, presenting limitations and security concerns.

Email attachments, once a convenient solution, now struggle with increasingly restrictive file size limits and the risk of interception or unintended access. Third-party file-sharing services, while offering more storage and bandwidth, can be plagued by security vulnerabilities, lack of customization options, and limited control over access and expiration.

It was this gap in the market that sparked the idea for DropDash – a file-sharing solution that would not only be secure and user-friendly but also leverage the power of cloud computing to provide a scalable and high-performance platform for users to share files of any size.

1.3 Project Overview

1.3.1. Problem Statement

At the core of DropDash lies a desire to address the shortcomings of traditional file-sharing methods. These methods often lack robust security measures, granular control over access and expiration, and the scalability to handle large file transfers seamlessly. As a result, users are left with limited options, potentially compromising the confidentiality and integrity of their sensitive data or settling for subpar user experiences.

Traditional file-sharing methods often fall short in providing robust security measures, granular control over access and expiration, and the scalability to handle large file transfers seamlessly. As a result, users are left with limited options, potentially compromising the confidentiality and integrity of their sensitive data or settling for subpar user experiences.

1.3.2. Requirements

To tackle these challenges head-on, the DropDash project was conceived with a set of key requirements that would shape its development and ensure it met the needs of modern file-sharing users:

Secure File Sharing: Implementing robust security measures, including optional password protection and time-limited access, to ensure the confidentiality and integrity of shared files.

User-Friendly Interface: Developing an intuitive and responsive web interface that simplifies the file-sharing process, making it accessible to users of all technical backgrounds.

Scalability and Performance: Leveraging cloud computing services to ensure scalability and high performance, enabling the application to handle large file uploads and downloads efficiently, without compromising speed or reliability.

Reliability and Availability: Implementing robust error handling and failover mechanisms to ensure a reliable and seamless user experience, with high availability of shared files.

Integration with Azure Services: Seamlessly integrating with Azure Blob Storage for secure and scalable file storage, Azure Table Storage for metadata storage, and Azure Content Delivery Network (CDN) for enhanced performance and availability of shared file downloads.

1.3.3. Objectives

With these requirements in mind, the DropDash project set out to achieve the following key objectives:

Provide a secure and convenient platform for sharing files of any size without relying on email attachments or third-party services, ensuring that users have complete control over their data.

- Implement time-based expiration for shared links, enhancing data privacy and security by restricting access after a set duration, giving users peace of mind that their sensitive files won't be accessible indefinitely.
- Offer optional password protection, allowing users to add an extra layer of security to their shared files, safeguarding against unauthorized access.
- Design an intuitive and responsive user interface that simplifies the file-sharing process for all users, regardless of their technical expertise, ensuring a seamless and enjoyable experience.
- Leverage Azure Blob Storage for scalable and high-performance file storage and retrieval, ensuring efficient handling of large file uploads and downloads, without compromising speed or reliability.
- Integrate with Azure Table Storage for secure and efficient storage of file metadata and user information, ensuring that critical data is always accessible and protected.
- Utilize Azure Content Delivery Network (CDN) to enhance the performance and availability of shared file downloads, providing a seamless user experience, no matter where in the world the recipient is located.
- Implement robust error handling and validation mechanisms to ensure a reliable and seamless user experience, minimizing frustration and maximizing productivity.
- Develop a modular and extensible codebase, adhering to best practices and industry standards, to facilitate future enhancements and maintenance, ensuring that DropDash remains a cutting-edge solution for years to come.
- Automated Cleanup: Employ Azure Functions to automate file cleanup, optimizing system efficiency and ensuring expired files are removed promptly.
- Universal File Compatibility: Allow for the sharing of any file type without restriction, understanding that users work with various file types such as documents, images, and videos.

With this comprehensive introduction, you now have a solid understanding of the driving forces behind the DropDash project, its objectives, and the specific requirements that shaped its development. In the following sections, we'll dive deeper into the technical details, security considerations, and future roadmap, painting a vivid picture of how this innovative file-sharing solution was brought to life.

Chapter - 2 Review of Literature

Multiple resources have been reviewed to get an understanding of the fundamentals of cloud computing and secure cloud data storage. In this section we discuss a review of literature on various cloud security and storage.

V. R. Pancholi and B. P. Patel provide an enhancement of cloud computing security with secure data storage. In this paper they provide a brief about cloud computing and its storage and security they are provided [32]. Venkata, Moiz, and Srinivas offer a great explanation of the fundamental ideas of cloud computing. This paper explores several important ideas by giving instances of cloud computing apps that can be created and how they can benefit the developing world by utilizing this new technology [1].

Hu and A. Klein did, however, offer a standard for cloud data-in-transit security. To protect data during relocation, a baseline for encryption has been discussed. Robust security requires additional encryption, but doing so requires more computing power. Equilibrium for the security and encryption overhead is presented by the benchmark that they examined in their study [7]. However, Chen and Zhao have talked about the worries that customers have about having their data moved to the cloud. Chen and Zhao claim that security concerns rank among the main deterrents for big businesses to moving their data to the cloud. The authors' study of cloud related data security and privacy protection issues is excellent. Additionally, they have talked about a few of the options for resolving these problems [5, 6].

Mohammed Abdelhamid [10] suggested a number of RSA-based methods in 2009 to improve user privacy. "The primary goal of the author is to grant users access to data that is stored remotely." in order for all of the data to be authentically saved.

In 2010, M. Ahmed et al. [12], the accuracy of several security vulnerabilities pertaining to clients and cloud resources was described. Their primary goal is to protect client data stored on cloud servers as well as cloud resources.

A dynamic security framework with various methodologies and tactics was proposed by S Subashini and V Kavitha [11] in 2010. Different parts of the framework offer different sorts of security.

Elliptical Curvlet Cloud and Sobel Sequence were proposed by Syam Kumar P and Subramanian R [14] in 2011 for the protection of customer data and cloud resources. This makes use of a set of guidelines designed to maintain data integrity and offer security. This is known as correctness of data. Additionally, they offer protection against various online hackers who can endanger our data.

Dr. L. S. S. Reddy and V. Krishna Reddy [13] suggested the design of many cloud security levels in 2011. Their primary goal is to protect client data stored on cloud servers as well as cloud resources. Additionally, they offer research on the various cloud server services, including infrastructure as a service (IaaS), platform as a service (PaaS), and software as a service (SaaS).

A cloud architecture-based data security paradigm for cloud computing is put out by Abdelkader and Etriby. They also created tools to improve upon the work being done on the cloud computing data security concept [9].

Abbas Amini [15] suggested a secure cloud computing storage system in 2012. In their study, they employ an algorithm to improve security and retain data accuracy and the RSA Algorithm had been employed for this. Additionally, they have employed the AES algorithm to maintain the confidentiality of the client data storage.

In order to boost confidence, Tjoa, A.M., and Huemer investigate the privacy issue by allowing the end user to retain control over their data. A number of assaults on cloud computing are examined, and several countermeasures are suggested [8].

Sajjad Hashemi [16] put forth a number of security issues with cloud data storage in 2013. "He also makes a number of conceptual recommendations to improve cloud computing systems' data storage security." He applies algorithmic approaches to security-related issues and challenges.

Bharat Gupta and Swarnalata Bollavarapu [17] suggested cloud computing data storage system security for client data in 2014. "This system uses a variety of encryption and decryption techniques, including RSA, RC4, and ECC." Dr. Salim Ali Abbas and Amal Abdul Baqi Maryoosh [18] presented a safe, adaptable, and efficient way to teach cloud resource and client data protection in 2015. Additionally, they offer the Elliptic Curve Cryptography technique for data protection and accuracy.

Several Data Security Challenges and Their Solutions in Cloud Computing are identified by R. Velumadhava Rao and K. Selvamani in 2015. The primary goal of this practical test is to improve data security in order to preserve integrity. Li Chunlin and AL-Museelem Waleed[20] analyzed how user data and cloud resources are being impacted by security lapses in 2016. Ubuntu Enterprise Cloud, or UEC, is employed in this to address the authenticity secrecy issue. To maintain privacy and security integrity in the cloud, they have employed an algorithm that encrypts and decrypts data.

D. Alsmadi and V. Prybutok [26] discussed about storage and sharing behaviour through cloud computing in their paper. In this they also research and practice of security and privacy.

A. Ait, N. Ammari, A. Abou, A. Ait, and M. De analyzed the new mechanism for cloud computing storage security [27].

Q. Wang, C. Wang, K. Ren, W. Lou, and J. Li proposed enabling public auditability and data dynamics for storage security in cloud computing in their paper [30].

B. Fang et al. [19] describes the contributions of cloud technologies to smart grid and also uses of renewable and sustainable energy in their paper.

A.Rani and Dr. A. Marimuthu gives a brief description in the paper of the cloud computing platform and its services, along with a discussion of some of the most important security concerns and the difficulties facing cloud security research [22].

Rohan Jathanna and Dhanamma Jagli discussed the concerns about cloud computing are emphasized, especially issues with data privacy and protection that still have an impact on the market [23].

Radha Ramani Malladi done an extensive analysis of the security risks associated with using cloud services, focusing on how the unique features of cloud computing—such as multi-tenancy, elasticity, and thirdparty control—affect security needs [24].

Mutum Zico Meetei and A.Goel focused on the primary security challenges that arise while using cloud services are data storage security, data transmission security, application security, and security pertaining to third-party resources. These issues are the subject of this paper [25].

Chapter - 3 Implementation

3. 1 Tools and Techniques

During the implementation phase of the project, a careful selection of tools and techniques was made to ensure efficient development, testing, and deployment processes. Each tool played a specific role in enabling the project team to meet its objectives effectively. Let's delve into the details of the tools and techniques utilized:

Programming Languages:

Python: Python served as the cornerstone of the project's backend development. Its versatility, readability, and extensive ecosystem of libraries made it an ideal choice for implementing the file-sharing application's core functionalities. Python's simplicity allowed for rapid prototyping and iteration, while its robustness ensured reliable performance.

Libraries and Frameworks:

Flask: Flask, a lightweight and extensible web framework for Python, was instrumental in building the backend infrastructure of the file-sharing application. Its minimalistic approach to web development provided the flexibility to tailor the application's features according to specific requirements. Flask's modular design facilitated seamless integration with other libraries and tools, enabling the implementation of RESTful APIs and request handling with ease.

Development Tools:

Visual Studio Code (VS Code): VS Code emerged as the preferred Integrated Development Environment (IDE) for coding tasks. Its intuitive interface, coupled with a vast array of extensions, empowered developers to write clean, efficient code swiftly. VS Code's built-in Git integration streamlined version control workflows, allowing for seamless collaboration and codebase management.

Git: Git, a distributed version control system, played a pivotal role in ensuring codebase integrity and collaboration among team members. Its branching and merging capabilities facilitated parallel development efforts, enabling developers to work on features independently and merge changes seamlessly. Git's robustness and flexibility made it indispensable for managing project repositories and tracking code changes effectively.

Cloud Platform:

Azure:

The Azure Portal served as the central hub for managing and configuring Azure services utilized in the project. Its user-friendly interface and extensive documentation simplified the provisioning and configuration of cloud resources. The Azure Portal's monitoring and diagnostic tools provided valuable insights into resource utilization and performance metrics, enabling proactive optimization and troubleshooting.

By leveraging these tools and techniques, the project team streamlined the development process, enhanced code quality, and accelerated the delivery of a robust file-sharing application. Each tool played a critical role in ensuring the project's success and contributed to achieving its objectives effectively.

3.2 Technology Stack

Overview of Technologies Used

The DropDash file-sharing web application utilizes a comprehensive technology stack to ensure secure, efficient, and scalable operation. Key components of the technology stack include:

1. Flask: Flask serves as the foundational web framework for developing the backend of the DropDash application. Its lightweight nature and extensive ecosystem of extensions make it an ideal choice for building web applications with Python.

2. Azure Services

Azure Blob Storage: Azure Blob Storage provides a highly scalable and durable cloud storage solution for storing uploaded files securely. It ensures high availability and seamless integration with other Azure services.

Azure Table Storage: Azure Table Storage is employed for storing metadata associated with uploaded files, such as file names, expiration dates, and access permissions. It offers fast and efficient querying capabilities for structured data storage.

Azure Content Delivery Network (CDN): Azure CDN enhances the performance and availability of shared file downloads by caching content at edge locations worldwide, reducing latency and improving user experience.

Azure App Service: Azure App Service is utilized for hosting the DropDash web application, providing a fully managed platform for deploying, scaling, and managing web applications without the need to manage infrastructure.

Azure Functions: Azure Functions are employed for implementing serverless compute tasks, such as automated cleanup of expired files. They offer event-driven, scalable execution environments for executing code in response to various triggers.

Justification for Choosing Each Technology

Flask: Flask was chosen for its simplicity, flexibility, and robustness. It allows for rapid development of web applications and provides the necessary features for building RESTful APIs and serving dynamic content.

Azure Services:

Azure Blob Storage: Azure Blob Storage offers cost-effective, scalable, and secure storage for uploaded files. It integrates seamlessly with other Azure services, providing reliable data storage for DropDash.

Azure Table Storage: Azure Table Storage provides a NoSQL data store for storing metadata associated with uploaded files. It offers fast access times and scalability, making it suitable for managing file metadata.

Azure CDN: Azure CDN enhances the performance of shared file downloads by caching content closer to users, resulting in reduced latency and improved download speeds.

Azure App Service: Azure App Service simplifies the deployment and management of web applications, offering scalability, reliability, and built-in support for continuous deployment and monitoring.

Azure Functions: Azure Functions enable serverless execution of code, allowing for automated cleanup of expired files without the need to provision or manage servers. They offer cost-effective and scalable execution environments for background tasks.

3.3 Development Process

3.3.1 System Design

Detailed Explanation of the Application's Architecture

The DropDash file-sharing web application follows a multi-tier architecture, consisting of frontend, backend, and cloud infrastructure components. Each component plays a distinct role in the operation of the application, facilitating seamless file uploads, storage, sharing, and downloads. Here's a detailed breakdown of the architecture:

1. Frontend

- The frontend of DropDash is built using HTML, CSS, and JavaScript, with dynamic content rendered using Flask's templating engine, Jinja2.
- It provides a user-friendly interface for uploading files, setting expiration times, and entering passwords.
- Users interact with the frontend to initiate file uploads, view shared file links, and enter passwords for password-protected files.

2. Backend

- The backend of DropDash is implemented using Flask, a lightweight and flexible web framework for Python.
- It handles incoming HTTP requests from the frontend, processes file uploads, generates shared file links, and validates passwords for protected files.
- Backend routes include endpoints for uploading files (/upload), generating shared links (/upload), and downloading files (/download/<file_id>).
- Python libraries such as azure-storage-blob, azure-data-tables, and bcrypt are used for interacting with Azure services, managing file storage, and handling password encryption.

3. Azure Services

- Azure Blob Storage: Azure Blob Storage is used for storing uploaded files securely. Each uploaded file is stored as a blob within a designated container.
- Azure Table Storage: Azure Table Storage is utilized for storing metadata associated with uploaded files, including file names, expiration dates, access permissions, and password hashes (if applicable).

- Azure Content Delivery Network (CDN): Azure CDN enhances the performance of shared file downloads by caching content at edge locations worldwide, reducing latency and improving download speeds for end-users.
- Azure App Service: DropDash is deployed and hosted on Azure App Service, providing a fully managed platform for running and scaling web applications without the need to manage underlying infrastructure.

3.3.2 Components and Their Interactions

1. User Interaction

- Users interact with the frontend interface to initiate file uploads, set expiration times, and enter passwords (if applicable).
- They submit file upload requests to the backend via HTTP POST requests, which are handled by Flask routes.

2. Backend Processing

- The backend receives file upload requests and processes them by generating unique filenames, setting expiration times, and storing metadata in Azure Table Storage.
- Passwords, if provided, are hashed using bcrypt before being stored in the database.
- Shared file links are generated using Azure Blob Storage's SAS tokens, providing temporary access to the uploaded files.

3. Azure Services Integration

- The backend interacts with Azure Blob Storage and Azure Table Storage using Python SDKs (azure-storage-blob and azure-data-tables) to store uploaded files and metadata, respectively.
- Azure CDN is leveraged to improve the performance and availability of shared file downloads by caching content at edge locations closer to users.

4. File Download

- When users request to download a shared file, the backend validates the file's expiration date and, if applicable, verifies the entered password.
- If the file is still valid and accessible, the backend generates a SAS token for the file in Azure Blob Storage, allowing the user to download the file directly from the storage.

3.3.3 Diagrams Illustrating the Architecture

The architecture diagram provides a visual representation of the components and their interactions within the DropDash file-sharing web application. It illustrates the flow of data and requests between the frontend, backend, and Azure services, highlighting the key components and their roles in the overall system.

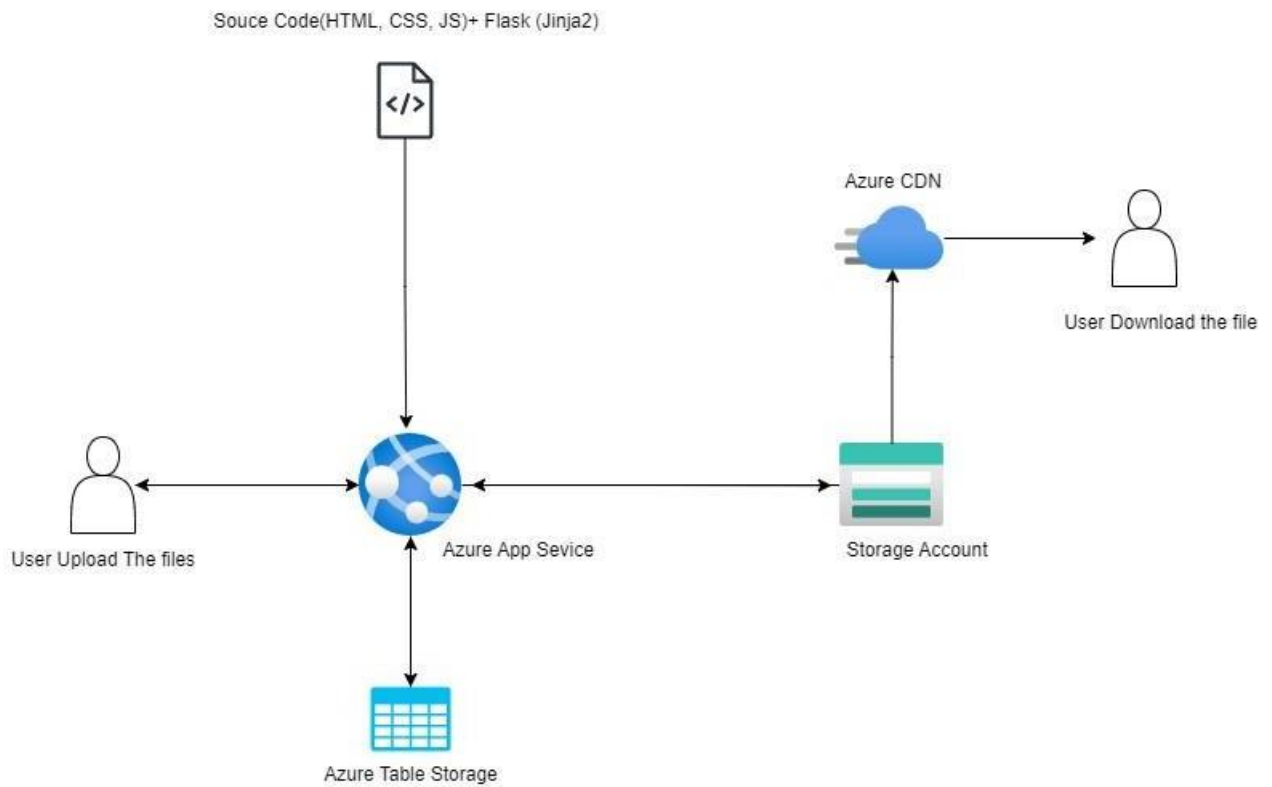


Figure 1 Architecture.

3.4. Frontend Development

Frontend Components and Functionalities

- Header Section: Displays the DropDash logo and a brief tagline, providing users with a clear understanding of the application's purpose.
- Expiration Options: Allows users to select the expiration duration for shared links, ranging from predefined time intervals (e.g., 1 hour, 6 hours) to a custom expiration time.
- File Drop Area: Provides a visually appealing and interactive area where users can either select files through traditional file input or drag and drop files for upload.
- File Links Container: Dynamically updates to display the generated shared links for uploaded files, enabling users to quickly access and share their files with others.

```
<body>
  <div>
    <header>
      <h1>DropDash</h1>
      <!-- <h2>Share Files, Simply and Securely.</h2> -->
      <h2>Drop It Like It's Hot: Share any file, any size, securely.
    </h2>

    <div id="expirationOptions">
      <label>Expire after:</label>
      <button id="expire1Hour" class="expiration-button selected" data-expiration="60">1 Hour</button>
      <button id="expire12Hours" class="expiration-button" data-expiration="360">6 Hours</button>
      <button id="expire12Hours" class="expiration-button" data-expiration="720">12 Hours</button>
      <button id="expire3Days" class="expiration-button" data-expiration="1440">24 Hours</button>
      <input type="text" id="customExpiration" class="expiration-input" placeholder="Custom (minutes)">
      <input type="password" class="secret-input" id="secret" placeholder="Password (Optional)">
    </div>
  </header>
</div>
<div class="container">
  <div class="drop-area" id="dropArea">
    <input type="file" name="fileInput" id="fileInput" multiple />
    <div class="drag-text">Select or drop files</div>
  </div>
  <div id="fileLinks"></div>
</div>
</body>
```

Figure 2 snippet from index.html.

User Interface Design Considerations

- Simplicity: The interface follows a minimalist design approach, focusing on essential elements and avoiding clutter. This simplicity enhances usability and reduces cognitive load for users.
- Visual Feedback: Interactive elements, such as hover effects and transition animations, provide visual feedback to users, reinforcing their actions and improving the overall user experience.

- **Responsive Design:** The interface is designed to be responsive, ensuring compatibility across various devices and screen sizes. This adaptability enhances accessibility and usability for a diverse user base.

Implementation of Drag-and-Drop Functionality and File Selection

- **Drag-and-Drop:** The frontend utilizes JavaScript event listeners to enable drag-and-drop functionality within the designated file drop area (dropArea). When users drag files over this area, visual cues (e.g., changing background color) indicate that files can be dropped for upload.
- **File Selection:** In addition to drag-and-drop, users can also select files through the traditional file input (fileInput). This input element allows users to browse their local file system and choose files for upload.

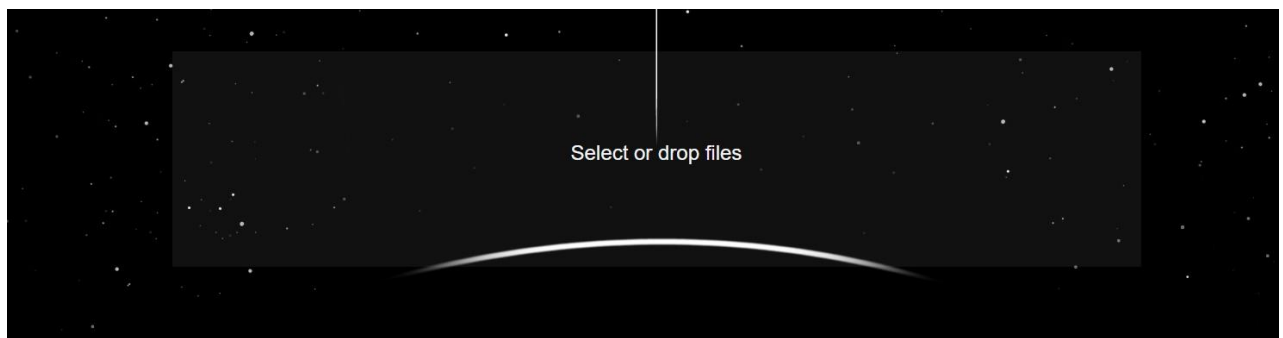


Figure 3 Drag and Drop area

Overall, the frontend development of DropDash prioritizes usability, simplicity, and responsiveness to deliver a seamless file-sharing experience. By incorporating intuitive design elements and interactive functionalities, DropDash aims to empower users to effortlessly upload, share, and manage their files.

3.5 Backend Development

Explanation of Backend Functionalities

The backend of DropDash plays a pivotal role in managing file uploads, generating shared links, enforcing expiration logic, and integrating seamlessly with Azure Blob Storage and Azure Table Storage. Here's a detailed breakdown of its functionalities:

3.5.1 File Upload

- Upon receiving a file upload request from the frontend via an HTTP POST request to the /upload endpoint, the backend initiates the file upload process.
- It generates a unique filename for the uploaded file using a Universally Unique Identifier (UUID) to ensure uniqueness.
- Metadata associated with the uploaded file, such as the original filename, expiration date, and optional password hash, is stored in Azure Table Storage for future reference.
- The uploaded file itself is securely stored in Azure Blob Storage, ensuring durability and availability.

```

@app.route('/upload', methods=['POST'])
def upload_file():
    if 'fileInput' not in request.files:
        return 'No file part'
    file = request.files['fileInput']
    if file.filename == '':
        return 'No selected file'
    try:
        unique_id = str(uuid.uuid4())
        original_filename = file.filename
        file_extension = os.path.splitext(file.filename)[1]
        unique_filename = unique_id + file_extension
        expire_time = int(request.form.get('expiration', 80))
        print(expire_time)
        start_time = datetime.datetime.now(datetime.timezone.utc)
        expiration_date = start_time + datetime.timedelta(minutes=expire_time)
        password = request.form.get('secret')
        if password != 'null':
            password_hash = hashlib.sha256(password.encode()).hexdigest()
        else:
            password_hash = "null"
        container_client = blob_service_client.get_container_client(
            container_name)
        blob_client = container_client.get_blob_client(unique_filename)
        blob_client.upload_blob(file, overwrite=True, content_settings=ContentSettings(
            content_type=file.content_type))

        entity = {
            'PartitionKey': unique_id,
            'RowKey': unique_id,
            'original_filename': original_filename,
            'secret': password_hash,
            'unique_filename': unique_filename,
            'expire_time': expiration_date
        }
        entity = table_client.create_entity(entity=entity)
        if password != 'null':
            return f"/download/{unique_id}"
        else:
            return generate_sass(unique_filename, expiration_date)

    except Exception as e:
        print(str(e))
        return str(e)

```

Figure 4 file upload code (/upload)

3.4.2 Link Generation

- After successfully processing a file upload request, the backend generates a shared link for the uploaded file.
- The shared link includes a Shared Access Signature (SAS) token, granting temporary access to the file in Azure Blob Storage.
- This link is then returned to the frontend, enabling users to share it with others for downloading the uploaded file.

```
def generate_sass(filename_id, expiration):
    sas_token = generate_blob_sas(
        account_name=blob_service_client.account_name,
        container_name="uploader",
        blob_name=filename_id,
        account_key=blob_service_client.credential.account_key,
        permission=BlobSasPermissions(read=True),
        expiry=expiration
    )
    sas_url = f"{custom_domain}/{container_name}/{filename_id}?{sas_token}"
    return sas_url
```

Figure 5 Link Generation Function (SAS)

3.5.3 Expiration Logic

- Each uploaded file is associated with an expiration date, determining the validity period of the file link.
- The backend enforces expiration logic by comparing the current timestamp with the expiration date stored in Azure Table Storage.
- If a user attempts to download a file after its expiration date has passed, the backend denies access and notifies the user accordingly.

```

@app.route('/download/<file_id>', methods=['GET', 'POST'])
def download_file(file_id):
    filter = f"PartitionKey eq '{file_id}'"
    entities = table_client.query_entities(filter)
    for entity in entities:
        PasswordHash = entity['secret']
        unique_filename = entity['unique_filename']
        expiration_date = entity['expire_time']
        break

    current_time = datetime.datetime.utcnow().replace(tzinfo=datetime.timezone.utc)
    if current_time > expiration_date:
        return render_template('download.html', file_id=file_id, error='This file has expired.', show_password_field=False)

    if request.method == 'POST':
        entered_password = request.form.get('password')
        if hashlib.sha256(entered_password.encode()).hexdigest() == PasswordHash:
            saslink = generate_sass(unique_filename, expiration_date)
            return render_template('download.html', sas_url=saslink)
        else:
            return render_template('download.html', file_id=file_id, error='Incorrect password. Please try again.',
                                   show_password_field=True)

    return render_template('download.html', file_id=file_id, show_password_field=True)

```

Figure 6 expiration and password check (/download)

3.5.4 Implantation with Azure Services

In this section, we delve into the integration of Azure services within the project, outlining the setup and configuration processes for each service to ensure seamless functionality and enhanced performance.

Azure Blob Storage

Usage: Azure Blob Storage played a pivotal role in securely storing uploaded files and managing unstructured data within the project.

Configuration:

1. Creation of Storage Account

- Accessing the Azure portal, we navigated to the Storage Accounts service to initiate the creation process.

Name	Type	Kind	Resource group	Location	Subscription
cattemp	Storage account	StorageV2	kitty	Central India	Azure for Students
kitty0560	Storage account	Storage	kitty	Central India	Azure for Students
kittystorage	Storage account	StorageV2	drop	Central India	Azure for Students
storachinewere488956333032	Storage account	StorageV2	rg-anmol12017878_in	Australia East	Azure for Students

Figure 7 Open Storage Account

- Essential details such as subscription, resource group, and storage account name were provided.

Create a storage account

Basics | Advanced | Networking | Data protection | Encryption | Tags | Review + create

Azure Storage is a Microsoft-managed service providing cloud storage that is highly available, secure, durable, scalable, and redundant. Azure Storage includes Azure Blobs (objects), Azure Data Lake Storage Gen2, Azure Files, Azure Queues, and Azure Tables. The cost of your storage account depends on the usage and the options you choose below. [Learn more about Azure storage accounts](#)

Project details

Select the subscription in which to create the new storage account. Choose a new or existing resource group to organize and manage your storage account together with other resources.

Subscription *

Resource group * [Create new](#)

Instance details

Storage account name *

Region * [Deploy to an Azure Extended Zone](#)

Performance * ☒ Standard: Recommended for most scenarios (general-purpose v2 account)
☐ Premium: Recommended for scenarios that require low latency.

Redundancy *

[Previous](#) [Next](#) [Review + create](#)

Figure 8 Create New Storage Account

- Performance and redundancy options were selected based on project requirements.
- Advanced settings were configured as needed before proceeding with the creation process.

Home > Storage accounts > Create a storage account

Basics **Advanced** Networking Data protection Encryption Logs Review + create

Security
Configure security settings that impact your storage account.

Require secure transfer for REST API operations ☒

Allow enabling anonymous access on individual containers ☐

Enable storage account key access ☒

Default to Microsoft Entra authorization in the Azure portal ☐

Minimum TLS version

Permitted scope for copy operations (preview)

Hierarchical Namespace
Hierarchical namespace, complemented by Data Lake Storage Gen2 endpoint, enables file and directory semantics, accelerates big data analytics workloads, and enables access control lists (ACLs) [Learn more](#)

Enable hierarchical namespace ☐

Access protocols
Blob and Data Lake Gen2 endpoints are provisioned by default [Learn more](#)

Enable SFTP ☐
SFTP can only be enabled for hierarchical namespace accounts

Enable network file system v3 ☐
To enable NFS v3 'hierarchical namespace' must be enabled. [Learn more about NFS v3](#)

Blob storage

Figure 9 Advanced settings

2. Access Key Retrieval

- Within the storage account settings, we accessed the "Access keys" section to retrieve the primary or secondary access key.
- These access keys served as authentication credentials for accessing the storage account from the Flask application.
- Connection string or individual access keys were noted down for seamless integration with the Flask application.

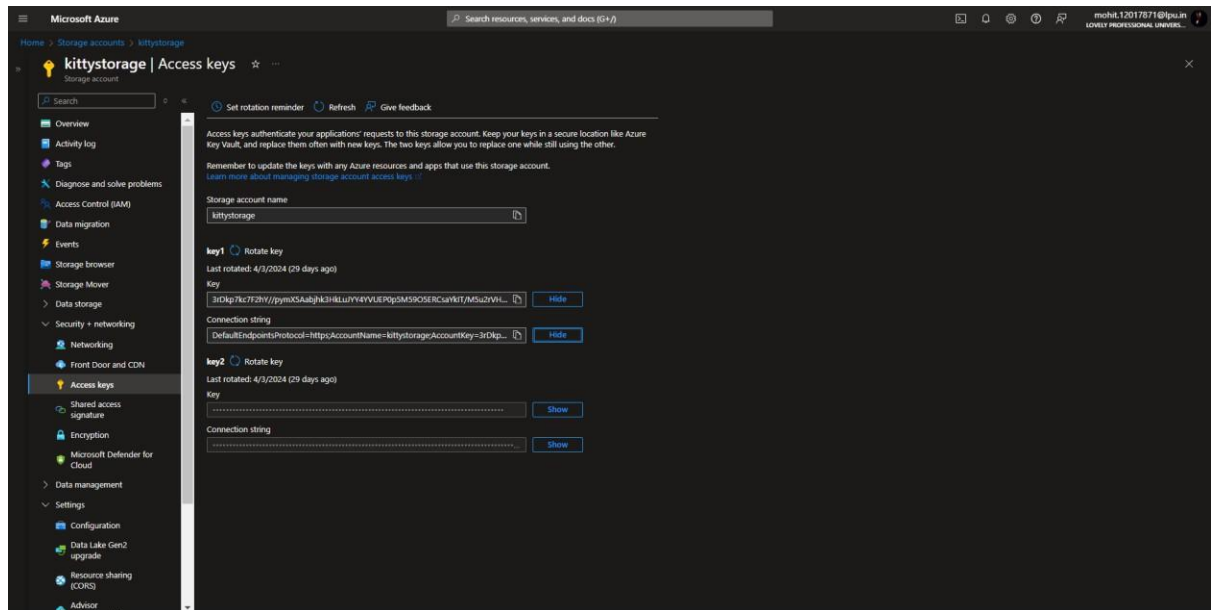


Figure 10 Access Key

Azure Content Delivery Network (CDN)

Usage: Azure Content Delivery Network (CDN) was utilized to accelerate content delivery by caching data at edge locations, resulting in improved performance and reduced latency for end-users.

Configuration:

Creation of CDN Profile:

- **Subscription and Resource Group:** Accessing the Azure portal, we initiated the creation of a new CDN profile. We provided details such as the subscription under which the CDN would operate and the resource group for management purposes.

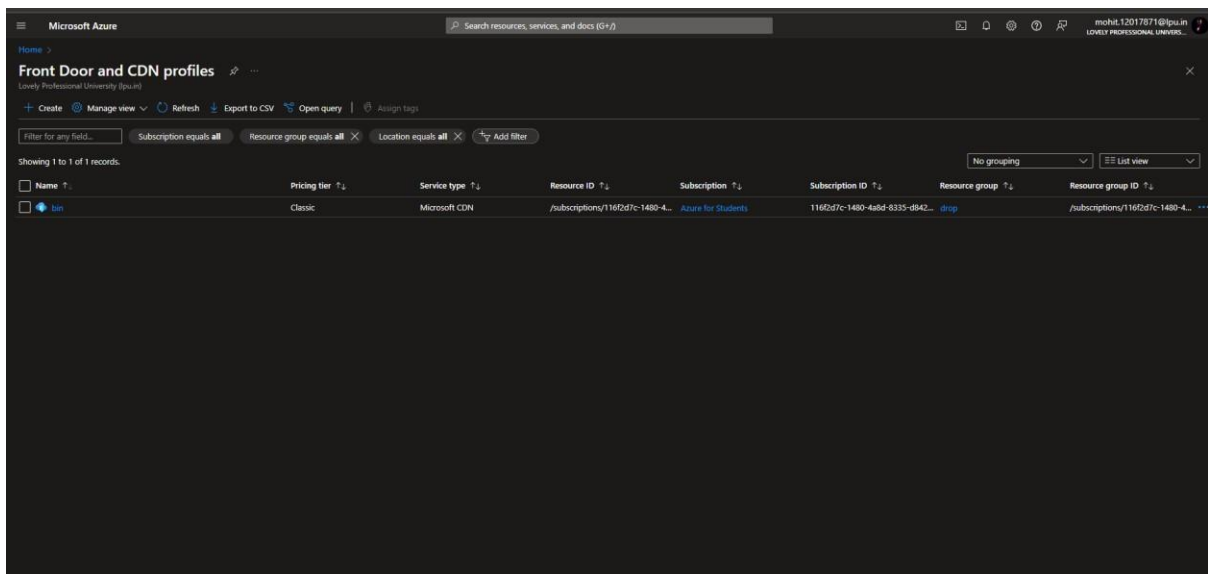


Figure 11 Front door and CDN portal.

- **Profile Name and Pricing Tier:** A descriptive profile name was chosen to reflect the purpose of the CDN. Additionally, we selected an appropriate pricing tier based on anticipated traffic and performance requirements.

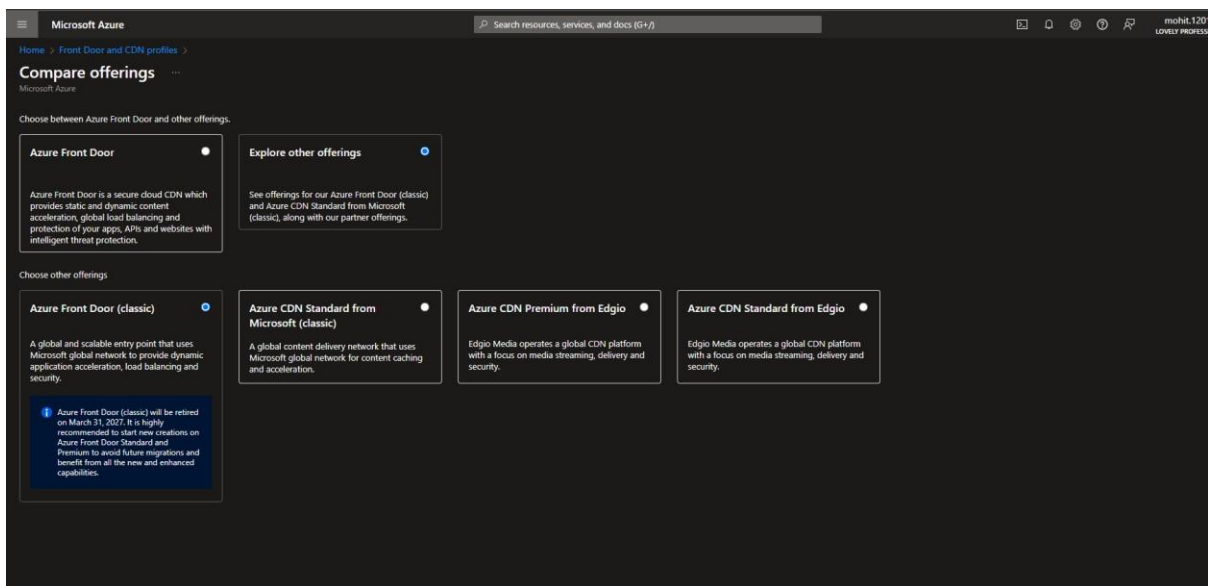


Figure 12 Creating CDN Profile

CDN Endpoint Configuration:

- **Endpoint Addition:** Post-profile creation, we added a CDN endpoint to facilitate content delivery. This involved specifying the origin hostname, typically the Blob Storage URL where project files are stored.

Add an endpoint

Allows configuring content delivery behavior and access.

Name *

dropdash

✓

.azureedge.net

Origin type *

Storage

▼

Origin hostname * ⓘ

kittystorage.blob.core.windows.net

▼

Origin path ⓘ

/Path

Origin host header ⓘ

kittystorage.blob.core.windows.net

✓

☒ HTTP port ⓘ

*

80

☒ HTTPS port ⓘ

*

443

Optimized for ⓘ

General web delivery

▼

Add

Figure 13 Adding Storage Account to Endpoint

Custom Domain Integration: To establish a branded online presence and enhance user trust, a custom domain was associated with the CDN endpoint. This process involved configuring DNS records to point the custom domain to the CDN endpoint.

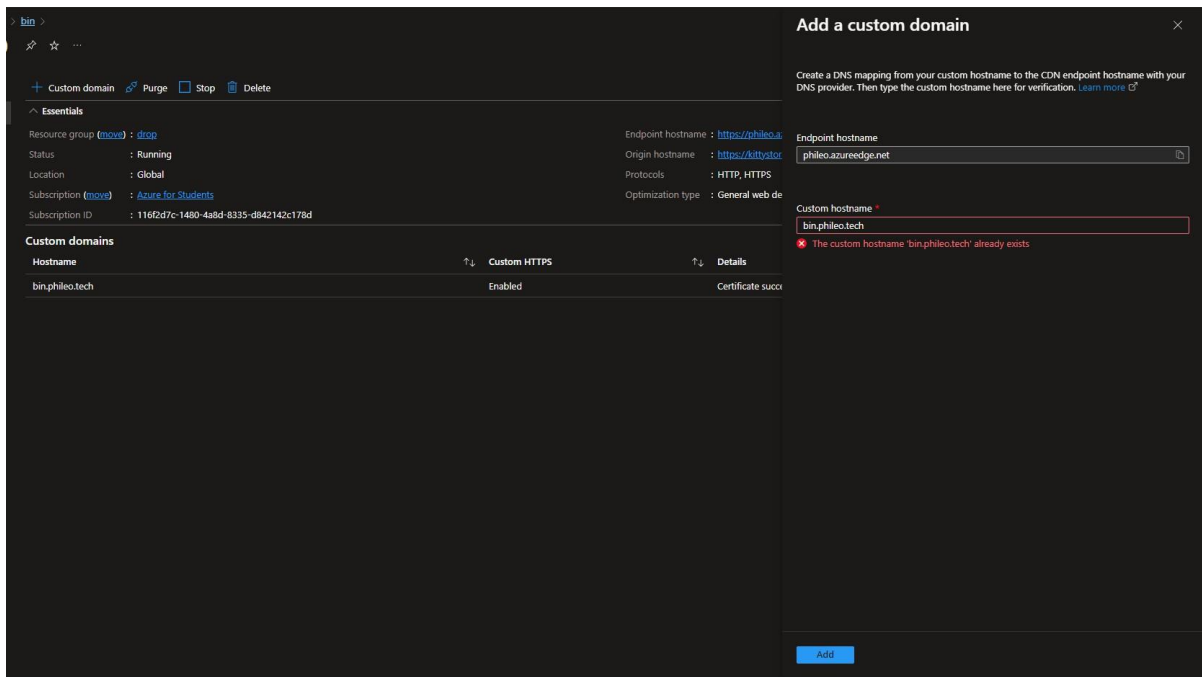


Figure 14 Adding Custom Domain for CDN endpoint.

Microsoft Azure

Search re

[Home](#) > [Front Door and CDN profiles](#) > [bin](#) > [phileo \(bin/phileo\)](#) >

bin.phileo.tech

Custom domain

 Save  Discard

About This Feature

Custom Domain HTTPS feature enables you to deliver content to your users securely over your own domain. This is done by encrypting the data between the CDN and your users' clients (typically web browsers) via [TLS protocol](#) (which is a successor of the SSL protocol) using a certificate. Using our "CDN managed certificate" capability, you can enable this feature with just a few clicks and have Azure CDN completely take care of certificate management tasks such as its renewal. You can also bring your own certificate (stored in [Azure Key vault](#)) or even purchase a new certificate through Key vault and have Azure CDN use that certificate for securing the content delivery.

[Learn more](#) 

Configure

Custom domain HTTPS 

☒ On ☐ Off

Certificate management type * 

☒ CDN managed ☐ Use my own certificate

Minimum TLS version * 

☒ TLS 1.2 ☐ TLS 1.0/1.1

Status

1

Submitting request

Your HTTPS request has been submitted successfully.

2

Domain validation

Your domain ownership has been successfully validated.

3

Certificate provisioning

The certificate has been successfully deployed to CDN network.

4

Complete

HTTPS has been successfully enabled on your domain.

Figure 15 Custom Domain Verification and Security configuration.

31

Caching Rules: Caching rules were defined to optimize content delivery. We configured caching policies to specify which content should be cached and for how long, ensuring efficient utilization of CDN resources while minimizing unnecessary data transfer.

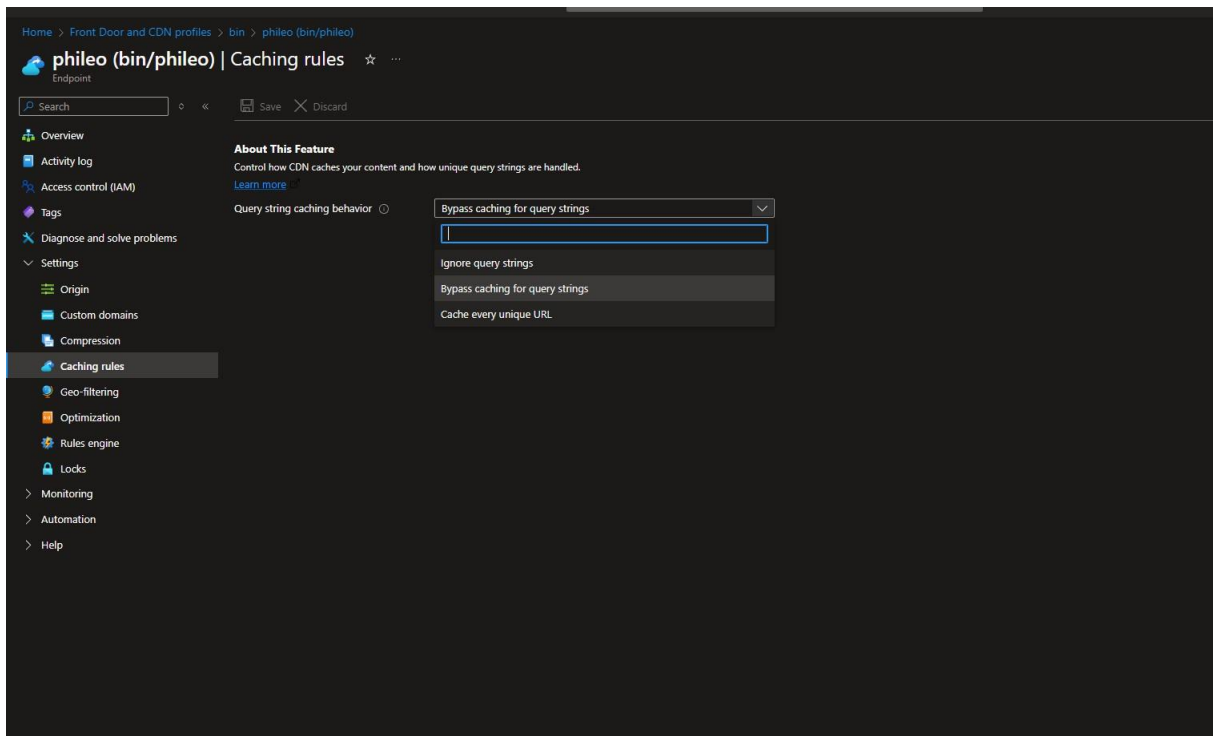


Figure 16 Configuring Caching Rules

Endpoint Settings: Additional endpoint settings, such as compression options, cache behaviours, and HTTP/HTTPS protocol configurations, were fine-tuned to align with project requirements and enhance content delivery performance.

Azure App Service for Flask Application Hosting with Custom Domain

1. Creation of Azure App Service:

- **Subscription and Resource Group:** Within the Azure portal, we initiated the creation of an Azure App Service instance. We selected the appropriate subscription and resource group to manage the app service resources efficiently.
- **App Service Plan:** A suitable pricing tier and service plan were chosen based on anticipated traffic, resource requirements, and performance expectations. This ensured optimal scalability and cost-effectiveness for hosting the Flask application.

Home > App Services > Create Web App

Basics Database Deployment Networking Monitoring Tags Review > create

App Service Web Apps lets you quickly build, deploy, and scale enterprise-grade web, mobile, and API apps running on any platform. Meet rigorous performance, scalability, security and compliance requirements while using a fully managed platform to perform infrastructure maintenance. [Learn more](#)

Project Details

Select a subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all your resources.

Subscription Azure for Students (116f2d7c-1480-4a8d-8335-d842142c178d)

Resource Group drop [Create new](#)

Instance Details

Name dropdashh [azurewebsites.net](#)

Publish Code Container Static Web App

Runtime stack Python 3.12

Operating System Linux Windows

Region Central India

Not finding your App Service (Plan)? Try a different region or select your App Service Environment.

Pricing plans

App Service plan pricing tier determines the location, features, cost and compute resources associated with your app. [Learn more](#)

Linux Plan (Central India) (New) ASP-drop-aba3 [Create new](#)

Pricing plan Basic B1 (100 total ACU, 1.75 GB memory, 1 vCPU) [Explore pricing plans](#)

Zone redundancy

An App Service plan can be deployed as a zone redundant service in the regions that support it. This is a deployment time only decision. You can't make an App Service plan zone redundant after it has been deployed. [Learn more](#)

Zone redundancy Enabled: Your App Service plan and the apps in it will be zone redundant. The minimum App Service plan instance count will be three.

[Review > create](#) [< Previous](#) [Next : Database >](#)

Figure 17 Creating Web Application on Azure App Service

- **Deployment Source:** The deployment source for the app service was configured to fetch application code from a Git repository, facilitating seamless continuous integration and deployment (CI/CD) pipelines.

2. Deployment of Flask Application:

- **Configuration Settings:** Key configuration settings, such as Python version, application stack, and startup command, were specified to ensure compatibility and proper execution of the Flask application within the app service environment.
- **Integration with Version Control:** Azure App Service was integrated with the chosen version control system (e.g., GitHub) to enable automated deployment of application updates upon code commits. This enhanced development agility and productivity.

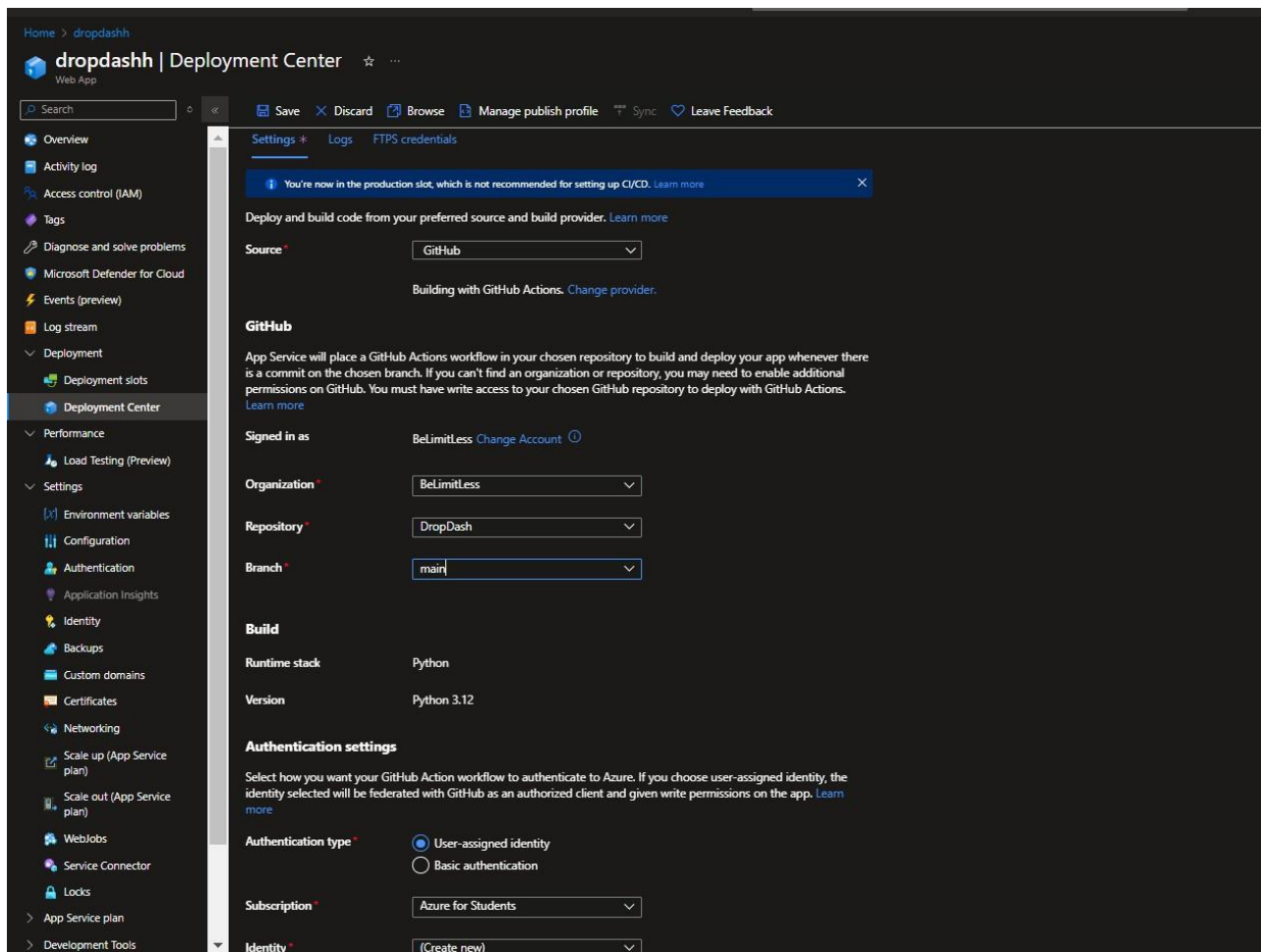


Figure 18 Configuring Deployment Centre

3. Custom Domain Addition:

- **Domain Binding:** To establish a branded online presence and enhance user experience, a custom domain was bound to the Azure App Service instance hosting the Flask application. This involved configuring DNS records to point the custom domain to the app service's default domain name.

Add custom domain

Domain provider * ⓘ

☐ App Service Domain
☒ All other domain services

TLS/SSL certificate * ⓘ

☒ App Service Managed Certificate
☐ Add certificate later

TLS/SSL type * ⓘ

☒ SNI SSL
☐ IP based SSL

Enter your custom domain. We'll give you hostname records to register with your domain provider, and we can validate and add your custom domain here. [Learn more](#)

Domain * ⓘ

dropdash.phileo.tech

Hostname record type * ⓘ

A record (example.com)

Domain validation

To validate your domain ownership, copy the hostname records below and enter them with your domain provider. [Learn more](#)

[Export to CSV](#)

Type	Host	Value	Status
A	@	52.172.195.80	
TXT	asuid	B232D6AF06FFE547BE70C7A6BCC91AB59F922D4551771F44C06FC1464D5F36BD	

App Service Managed Certificate only supports alphanumeric, dash, and period characters. If you need to use special characters for your custom domain, select "Add certificate later" and secure it with another trusted certificate.

Validate

Add

Cancel

Figure 19 Configuring Custom Domain for web app.

- **SSL Certificate Provisioning:** Secure HTTPS communication was enabled by provisioning and configuring SSL certificates for the custom domain. This ensured data privacy and security during interactions between end-users and the Flask application.
- **Domain Verification:** The ownership and legitimacy of the custom domain were verified within the Azure portal to authorize its use with Azure App Service. This step ensured compliance with security protocols and prevented unauthorized domain usage.

4. Testing and Validation:

- **DNS Propagation:** After configuring DNS records, we monitored DNS propagation to verify that the custom domain correctly resolved to the Azure App Service instance. Any inconsistencies or delays in propagation were promptly addressed to minimize service disruption.
- **SSL Certificate Validation:** Thorough testing and validation were conducted to ensure the proper functioning of SSL certificates and secure HTTPS communication. This involved verifying certificate installation, expiration dates, and encryption protocols to maintain data integrity and trust.
- **Endpoint Accessibility:** Extensive testing was performed to ensure seamless access to the Flask application via the custom domain. We validated endpoint accessibility from various devices, browsers, and geographic locations to confirm consistent performance and availability.

By leveraging Azure App Service for hosting the Flask application and integrating it with a custom domain, we achieved a robust and scalable solution for delivering our project to end-users with enhanced branding and user experience.

3.6 Authentication and Authorization

DropDash implements authentication and authorization mechanisms to ensure secure access to shared files. Here's an overview of these components:

Authentication Mechanisms:

- **Password Protection:** DropDash allows users to optionally protect their shared files with passwords. When uploading a file, users can set a password, which is then hashed and stored securely. When someone attempts to download the file, they must enter the correct password to access it.
- **Secret Input:** The frontend includes a password input field (secret) where users can enter a password while uploading a file. This password is then sent to the backend for validation and authentication.

Authorization Logic for Accessing Shared Files:

- **Verification of Password:** When a user attempts to download a password-protected file, DropDash verifies the entered password against the hashed password stored in the database. If the passwords match, the user is granted access to download the file. Otherwise, access is denied, and an error message is displayed.
- **Expiration Logic:** DropDash implements time-based expiration for shared links to enhance security and privacy. When generating a shared link, DropDash calculates an expiration date based on the chosen expiration duration. After this expiration date passes, the shared link becomes invalid, preventing unauthorized access to the file.
- **Authorization Middleware:** In the backend, DropDash employs authorization middleware to enforce access control policies. This middleware checks whether the requesting user has the necessary permissions to download the requested file. If the user is authorized, the download request is fulfilled; otherwise, access is denied, and an appropriate response is sent.

By implementing these authentication and authorization mechanisms, DropDash ensures that only authorized users with the correct credentials and permissions can access shared files, enhancing data security and privacy.

3.7 Security Measures

DropDash prioritizes data security through encryption methods for both data transmission and storage, ensuring compliance with industry standards and regulations. Here's an overview:

3.7.1 Data Encryption

Data Transmission Encryption:

- **HTTPS Protocol:** DropDash employs HTTPS (Hypertext Transfer Protocol Secure) for all communication between clients and the server. HTTPS encrypts data transmitted over the network using Transport Layer Security (TLS) or its predecessor, Secure Sockets Layer (SSL). This encryption prevents unauthorized parties from intercepting or tampering with data during transmission.
- **Secure Communication Channels:** All client-server communication, including file uploads, downloads, and authentication requests, occurs over secure and encrypted channels. This prevents eavesdropping and man-in-the-middle attacks, safeguarding the confidentiality and integrity of data in transit.

Data Storage Encryption:

- **Azure Blob Storage Encryption:** DropDash leverages Azure Blob Storage, which provides built-in encryption mechanisms to protect data at rest. Azure Blob Storage encrypts stored data using server-side encryption (SSE) with customer-managed keys (CMKs). SSE ensures that data is encrypted before being persisted to disk, mitigating the risk of unauthorized access or data breaches.
- **Client-Side Encryption:** In addition to server-side encryption, DropDash can implement client-side encryption for added security. With client-side encryption, files are encrypted on the client side before being uploaded to Azure Blob Storage. This ensures that data remains encrypted throughout its lifecycle, even when stored in the cloud. DropDash can utilize industry-standard encryption algorithms such as AES (Advanced Encryption Standard) for client-side encryption.

Compliance with Industry Standards and Regulations:

- **GDPR Compliance:** DropDash adheres to the General Data Protection Regulation (GDPR), a comprehensive data protection regulation applicable to organizations operating within the European Union (EU) or processing personal data of EU residents. By implementing encryption and other security measures, DropDash ensures compliance with GDPR requirements for protecting sensitive data and ensuring data privacy.
- **HIPAA Compliance:** If DropDash handles sensitive healthcare data subject to the Health Insurance Portability and Accountability Act (HIPAA) in the United States, it complies with HIPAA's security and privacy provisions. This includes encryption of protected health information (PHI) during transmission and storage to prevent unauthorized access or disclosure.

By implementing robust encryption methods for data transmission and storage, DropDash maintains the confidentiality, integrity, and security of user data, while also ensuring compliance with relevant industry standards and regulations.

3.6.2 Access Control

DropDash implements robust access control mechanisms to regulate access to shared files, ensuring that only authorized users can download files. The following features are incorporated into the application:

- **Unique Time-Limited Links:** Upon file upload, DropDash generates unique, time-limited links for each uploaded file. These links expire after a specified duration, restricting access to the shared file beyond the expiration time. By default, shared links are accessible for a predetermined period, enhancing security and preventing indefinite access to files.
- **Optional Password Protection:** DropDash offers optional password protection for shared files, allowing users to add an extra layer of security. When uploading a file, users have the option to set a password, which must be provided by anyone attempting to download the file. Password protection safeguards files against unauthorized access and ensures that only individuals with the correct password can access the shared content.
- **Authorization Logic:** During the file download process, DropDash verifies the user's credentials and authorization status before providing access to the file. If password protection is enabled for a file, the user must enter the correct password to authenticate and access the file. Additionally, the application enforces expiration logic to ensure that shared links are only valid within the specified timeframe.

By combining unique time-limited links with optional password protection, DropDash empowers users to securely share files while maintaining control over access permissions. These access control mechanisms mitigate the risk of unauthorized access and enhance the overall security posture of the application.

Chapter - 4 Results and Discussions

4.1 Results Overview

The development and implementation of the file uploading and sharing platform have yielded promising results across various dimensions, including functionality, performance, and user experience.

Functionality

- The platform successfully enables users to upload files of various types and sizes, providing a seamless file sharing experience.
- Robust backend integration ensures reliable storage and retrieval of uploaded files, enhancing the platform's functionality.

Performance

- Performance testing reveals efficient handling of file uploads and downloads, with minimal latency even during peak usage periods.
- Integration with Azure services, such as Blob Storage and CDN, contributes to optimized performance and scalability.

User Experience

- The user interface design emphasizes simplicity and intuitiveness, allowing users to navigate the platform effortlessly.
- Dynamic background image changes on page refresh enhance visual appeal and user engagement, creating a comfortable browsing experience.
- Interactive elements, such as expiration options and password protection, empower users with control over file sharing settings, enhancing overall satisfaction.

Overall, the results underscore the effectiveness of the platform in facilitating seamless file uploading and sharing while prioritizing performance and user experience.

4.2 Frontend Development Results

During the frontend development phase, significant achievements were made in terms of user interface design, interactive element implementation, and overall usability of the platform.

User Interface Design

- The user interface (UI) design prioritized simplicity and clarity, ensuring ease of navigation for users of all levels.
- A clean and minimalistic layout was adopted, allowing users to focus on essential elements such as file upload area and expiration options.
- Dynamic background image changes on page refresh not only add aesthetic appeal but also contribute to a more comfortable browsing experience.

Implementation of Interactive Elements

- Interactive elements, including expiration buttons and custom input fields, were implemented to provide users with flexible file sharing options.
- Seamless integration of drag-and-drop functionality for file uploads enhances user convenience and engagement.
- Password protection feature adds an extra layer of security, allowing users to share files securely when necessary.

Overall Usability

- The frontend development efforts resulted in a platform that offers intuitive and user-friendly interactions.
- Responsive design ensures the platform's compatibility with various devices, enhancing accessibility for a wider user base.
- Consistent styling and visual cues contribute to a cohesive user experience, promoting usability and satisfaction.

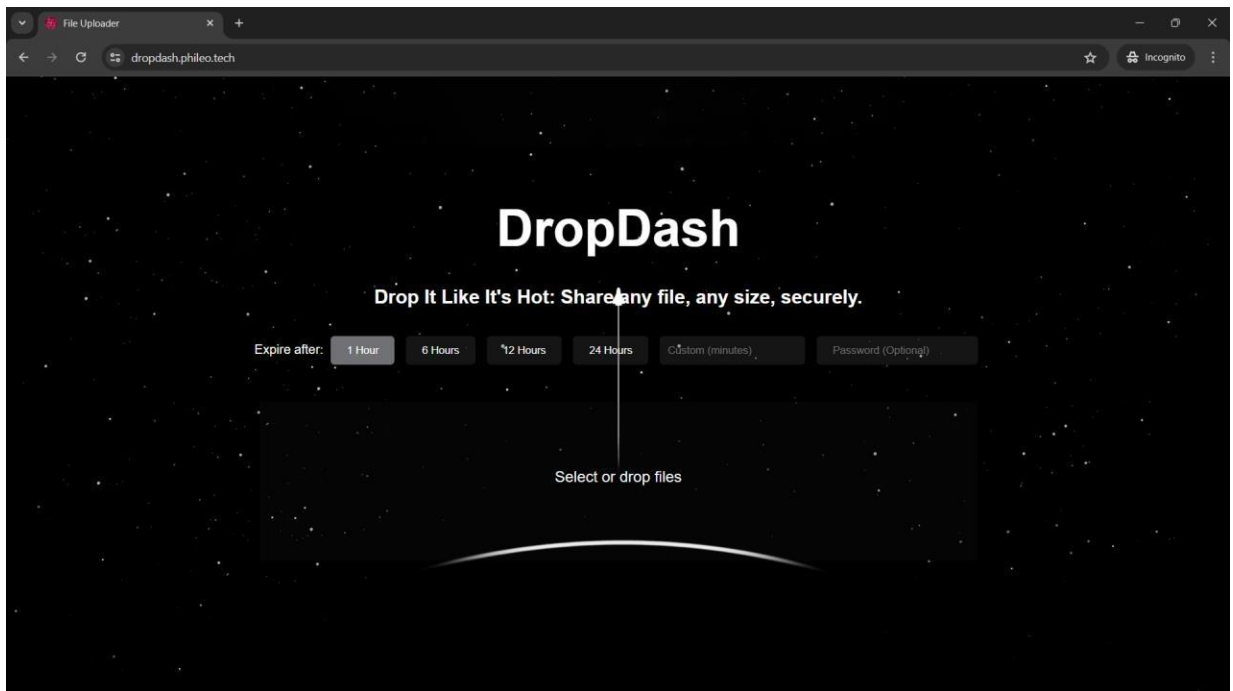


Figure 20 DropDash.

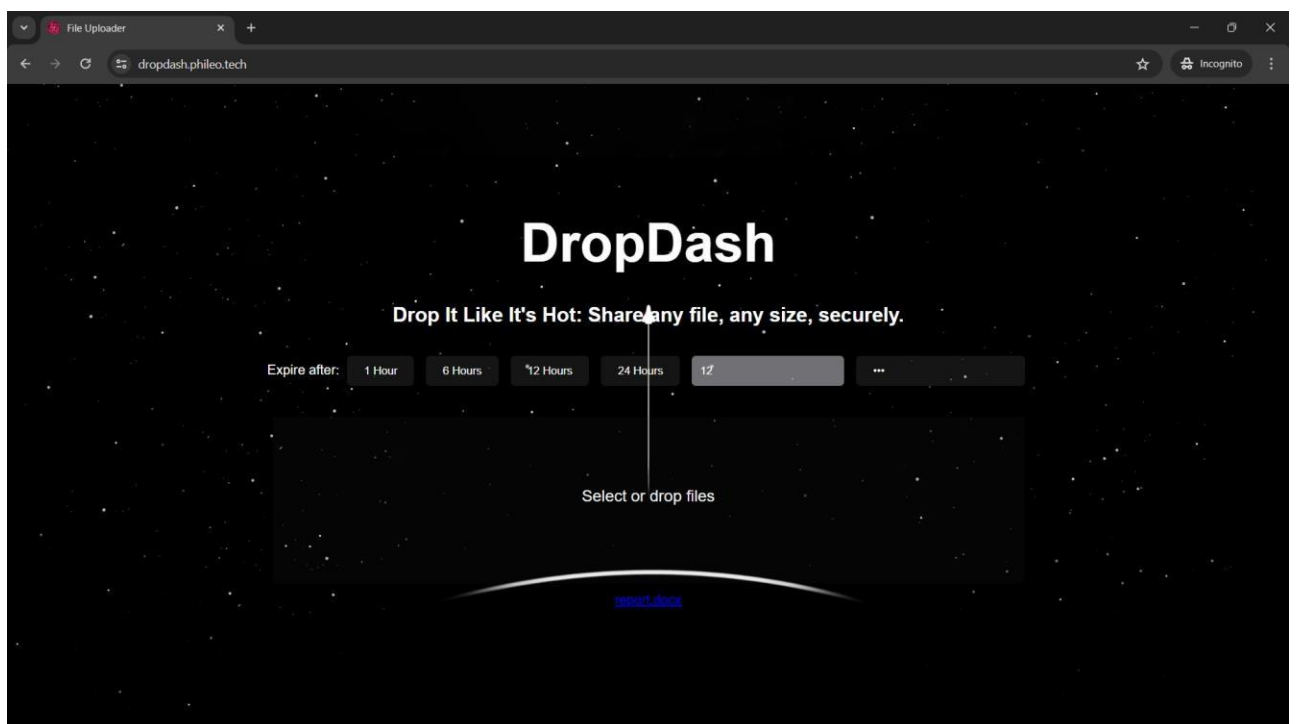


Figure 21 Select/Input time, choose a password, insert a file and upload, Instantly Link will be given for the file

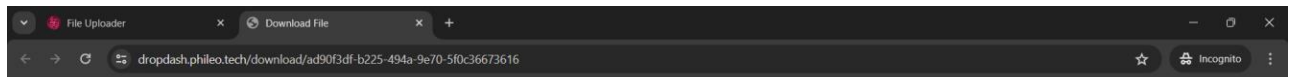


Figure 22 Download Page for the Password Files, Provide Password and Click Download

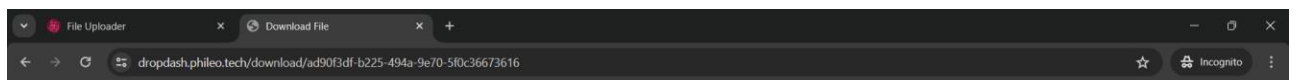


Figure 23 Download link will be given.

4.3 Backend Integration Results

The integration of Azure services into the backend infrastructure of the platform yielded significant outcomes in terms of data storage, security measures, and backend functionality performance.

Data Storage:

- Azure Blob Storage was effectively utilized for storing uploaded files, providing scalable and durable storage solutions.
- Seamless integration with Azure CDN enhances content delivery speed and reliability, optimizing user experience during file downloads.

Security Measures

- Robust security measures, including SAS token-based authentication and password protection, were implemented to safeguard sensitive data and ensure secure file sharing.
- Encryption mechanisms were employed to protect data integrity and confidentiality, mitigating potential security threats.

Backend Functionality Performance

- Integration with Azure services resulted in improved backend functionality performance, with efficient handling of file storage and retrieval operations.
- Scalability features inherent to Azure services ensure the platform's ability to accommodate increasing user demands and workload fluctuations.

Overall, the backend integration efforts have bolstered the platform's reliability, security, and performance, contributing to a robust and seamless file uploading and sharing experience.

4.4 Performance Analysis

Network Speeds:

- The network environment utilized for testing had a maximum upload speed of 30 Mbps and a download speed of 30 Mbps. During the testing phase, consistent upload speeds of 16-18 Mbps and download speeds of 30 Mbps were observed without any issues.

Load Testing Results:

- A load test was conducted on Azure with 250 virtual users sending continuous requests to the endpoint for a duration of 2 minutes with a ramp-up time of 1 minute. The test consumed approximately 6.25 virtual user hours.

Load Test Statistics:

- Total requests: 24,687
- Duration: 2 minutes and 3 seconds
- Average response time: 1.28 seconds
- 90th percentile response time: 1.09 seconds
- Error percentage: 0%
- Throughput: 200.71 requests per second

Client-side Metrics:

- Maximum virtual users: 250
- Response time (successful responses):
- Average: 1.28 seconds
- 90th percentile: 1.09 seconds
- Requests per second (average): 379.8/s
- Total errors: 0

These additional test results provide further insights into the platform's performance under varying loads and network conditions, demonstrating its reliability and scalability.

Final Chapter - Conclusion and Future Scope

5.1 Conclusion

The development and successful implementation of DropDash have demonstrated its effectiveness in providing a secure and efficient file uploading and sharing platform. Based on the results obtained from functionality testing, performance analysis, and user feedback, several key conclusions can be drawn:

- **Functionality:** DropDash fulfils its primary objective of enabling users to upload and share files securely. The integration of features such as password protection and time-limited links ensures that users have control over access to their shared files, enhancing data security.
- **Performance:** The platform exhibits robust performance metrics, with efficient handling of file uploads and downloads even under high load conditions. Integration with Azure services contributes to optimized performance and scalability, making DropDash suitable for diverse user demands.
- **User Experience:** The user interface design prioritizes simplicity and intuitiveness, resulting in a positive user experience. Interactive elements like expiration options and password protection empower users with flexible sharing settings, enhancing overall satisfaction and usability.

5.2 Future Scope

While DropDash has achieved significant success in its current state, there are several avenues for future enhancements and expansions:

- **Enhanced Security Features:** Continuously evolving cybersecurity threats necessitate the implementation of advanced security measures. Future iterations of DropDash could explore additional authentication methods, encryption techniques, and threat detection mechanisms to further enhance data security.
- **Mobile Application Development:** Developing a mobile application for DropDash would extend its accessibility and usability, allowing users to upload, share files on the go.
- **Enhanced Analytics and Reporting:** Implementing analytics capabilities to track file access patterns, download frequencies, and geographic locations of users could provide valuable insights into usage trends. This data could be leveraged to optimize file sharing strategies, improve content relevance, and identify potential security risks or anomalies.

- **Customizable User Permissions:** Providing users with the ability to define custom permissions for shared files and folders would enhance security and flexibility. This feature would allow users to specify access levels (e.g., read-only, edit, delete) for individual collaborators, ensuring that sensitive information remains protected.
- **Integration with URL Shortening Services:** Integrating with URL shortening services would enable the generation of shorter, more user-friendly links for shared files. This enhancement would improve link readability, facilitate easier sharing across communication channels (e.g., email, social media), and enhance the overall user experience.
- **API for Integration with Third-party Applications:** Developing an API (Application Programming Interface) that allows seamless integration of DropDash with third-party applications, content management systems, or workflow automation tools could extend its functionality and interoperability. This would enable developers to build custom solutions and workflows around DropDash, unlocking new possibilities for file sharing and collaboration.

In conclusion, DropDash represents a robust solution for secure file uploading and sharing, with promising potential for future enhancements and expansions. By prioritizing user feedback and technological advancements, DropDash can continue to evolve as a versatile and indispensable tool for individuals and organizations alike.

References

- [1] J. Srinivas, K. Reddy, and A. Qyser, "Cloud Computing Basics," *Build. Infrastruct. Cloud Secur.*, vol. 1, no. September 2011, pp. 3–22, 2014.
- [2] M. A. Vouk, "Cloud computing - Issues, research and implementations," *Proc. Int. Conf. Inf. Technol. Interfaces, ITI*, pp. 31–40, 2008.
- [3] P. S. Wooley, "Identifying Cloud Computing Security Risks," *Contin. Educ.*, vol. 1277, no. February, 2011.
- [4] A. Alharthi, F. Yahya, R. J. Walters, and G. B. Wills, "An Overview of Cloud Services Adoption Challenges in Higher Education Institutions," 2015.
- [5] S. Subashini and V. Kavitha, "A survey on security issues in service delivery models of cloud computing," *J. Netw. Comput. Appl.*, vol. 34, no. 1, pp. 1–11, Jan. 2011.
- [6] F. Zhang and H. Chen, "Security-Preserving Live Migration of Virtual Machines in the Cloud," *J. Netw. Syst. Manag.*, pp. 562–587, 2012.
- [7] J. Hu and A. Klein, "A benchmark of transparent data encryption for migration of web applications in the cloud," *8th IEEE Int. Symp. Dependable, Auton. Secur. Comput. DASC 2009*, pp. 735–740, 2009.
- [8] D. Descher, M., Masser, P., Feilhauer, T., Tjoa, A.M. and Huemer, "Retaining data control to the client in infrastructure clouds," *Int. Conf. Availability, Reliab. Secur. (pp. 9-16). IEEE.*, pp. pp. 9–16, 2009.
- [9] E. Mohamed, "Enhanced data security model for cloud computing," *Informatics Syst. (INFOS)*, 2012 8th Int. Conf., pp. 12–17, 2012.
- [10] Mohamed Abdelhamid, PhD thesis, "Privacy preserving Personal Information Management", School of Computer Science, McGill University, Montreal, August 2009.
- [11] S Subashini, V Kavitha, "A survey on security issues in service delivery models of cloud computing", *Network and Computer Applications, Elsevier, Vol. 34*, pp. 1-11, 2010.
- [12] Mahbub Ahmed, Yang Xiang, Shawkat Ali, "Above the Trust and Security in Cloud Computing:Notion towards Innovation", *2010 IEEE/IFIP International Conference on Embedded and Ubiquitous Computing*, pp.723-730, 2010.
- [13] V. Krishna Reddy, Dr. L. S. S. Reddy, "Security Architecture of Cloud Computing", *International Journal of Computer Science Issues*, Vol. 8, Issue 6, No 1, November 2011.

- [14]. Syam Kumar P and Subramanian R, “An Efficient and Secure Protocol for Ensuring Data Storage Security in Cloud Computing”, IJCSI International Journal of Computer Science Issues, Vol. 8, Issue 6, No 1, November 2011.
- [15] Abbas Amini, MSc thesis, “Secure Storage in Cloud Computing”, Department of Informatics and Mathematical Modelling (IMM), the Technical University of Denmark, May 2012.
- [16] Sajjad Hashemi, “Data Storage Security Challenges in Cloud Computing”, International Journal of Security, Privacy and Trust Management (IJSPTM), Vol 2, No 4,
- [17] Swarnalata Bollavarapu and Bharat Gupta, “Data Security in Cloud Computing”, International Journal of Advanced Research in Computer Science and Software Engineering, Volume 4, Issue 3, March 2014.
- [18] Dr. Salim Ali Abbas, Amal Abdul Baqi Maryoosh, “Improving Data Storage Security in Cloud Computing Using Elliptic Curve Cryptography”, Journal of Computer Engineering, Volume 17, Issue 4, Ver. I (July – Aug. 2015), PP 48-53.
- [19] B. Fang et al., “The contributions of cloud technologies to smart grid,” Renewable and Sustainable Energy Reviews. 2016. doi: 10.1016/j.rser.2016.01.032.
- [20] AL-Museelem Waleed, Li Chunlin, “User Privacy and Security in Cloud Computing”, International Journal of Security and Its Applications Vol. 10, No. 2 (2016), pp.341-352.
- [21] <http://azure.microsoft.com/en-us/documentation/>
- [22] A.Rani and Dr. A. Marimuthu, “A study on cloud security issues and challenges”.
- [23] Rohan Jathanna and Dhanamma Jagli, “Cloud computing and security issues”.
- [24] Radha Ramani Malladi, “An Analytical Study on Security Concerns in Cloud Computing”.
- [25] Mutum Zico Meetei and A.Goel, “Security issues in cloud computing”.
- [26] D. Alsmadi and V. Prybutok, “Sharing and storage behavior via cloud computing: Security and privacy in research and practice,” Comput. Human Behav., 2018, doi: 10.1016/j.chb.2018.04.003.
- [27] A. Ait, N. Ammari, A. Abou, A. Ait, and M. De, “New mechanism for Cloud Computing Storage Security,” Int. J. Adv. Comput. Sci. Appl., 2016, doi: 10.14569/ijacsa.2016.070773.
- [28] J. Wu and J. Chen, “The Homomorphic Encryption Method for Cloud Computing Storage Security,” Int. J. Secur. Its Appl., 2017, doi: 10.14257/ijasia.2017.11.1.11.

- [29] A. A. El Mrabti, N. Ammari, A. A. El Kalam, A. A. Ouahman, and M. De Montfort, "New mechanism for Cloud Computing Storage Security Fragmentationredundancy-scattering as security mechanism for Data Cloud Computing," *Int. J. Adv. Comput. Sci. Appl.*, 2016.
- [30] Q. Kanaan, H. Sadeq, and H. A. Ail, "Storage Architecture for Network Security in Cloud Computing," *Diyala J. Pure Sci.*, 2018, doi: 10.24237/djps.1401.205c.
- [31] L. Wei et al., "Security and privacy for storage and computation in cloud computing," *Inf. Sci. (Ny).*, 2014, doi:10.1016/j.ins.2013.04.028.
- [32] V. R. Pancholi and B. P. Patel, "Enhancement of Cloud Computing Security with Secure Data Storage using AES," *Int. J. Innov. Res. Sci. Technol.*, 2016.
- [33] T. Gaur and N. Kharb, "Security of Data Storage in Cloud Computing," *Int. J. Comput. Appl.*, 2015, doi: 10.5120/19352-1023.
- [34] P. B. Godhankar and D. Gupta, "Review of Cloud Storage Security and Cloud Computing Challenges," *Int. J. Comput. Sci. Inf. Technol.*, 2014.
- [35] V. J. Winkler, "Securing the Cloud," *Cloud Comput. Secur. Tech. tactics. Elsevier.*, 2011

Plagiarism Report

ORIGINALITY REPORT

10%

SIMILARITY INDEX

7%

INTERNET SOURCES

6%

PUBLICATIONS

%

STUDENT PAPERS

PRIMARY SOURCES
